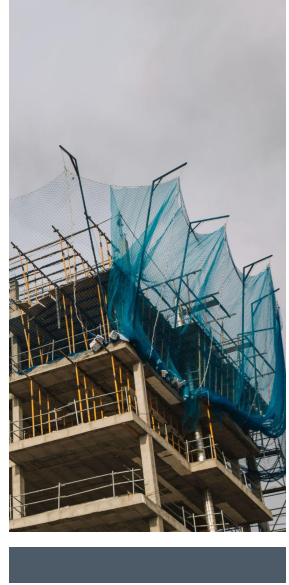
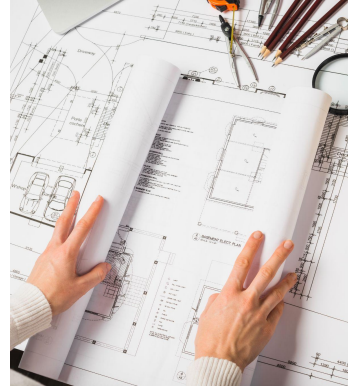
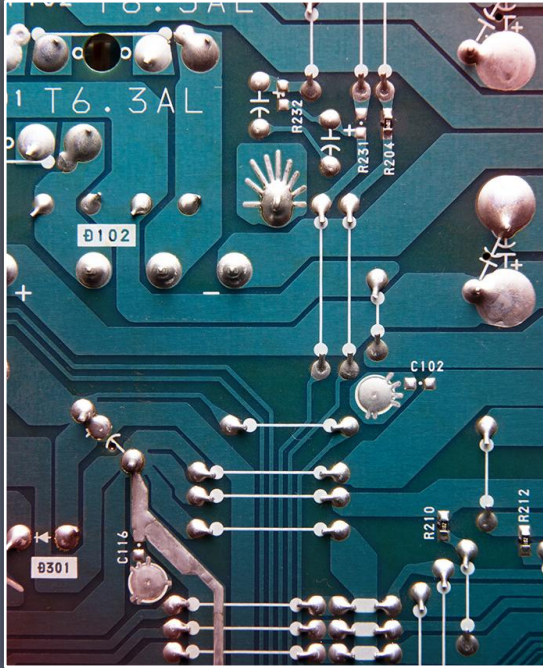


Software Architecture

Software Engineering Course – 3
2025-2026



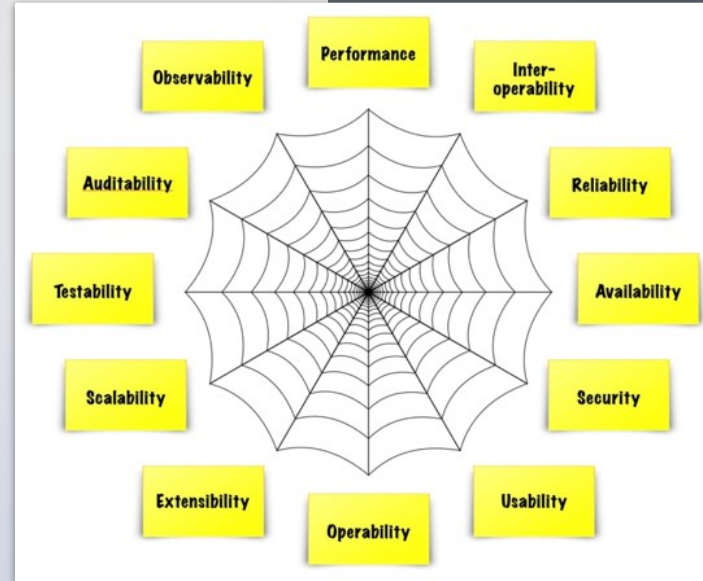


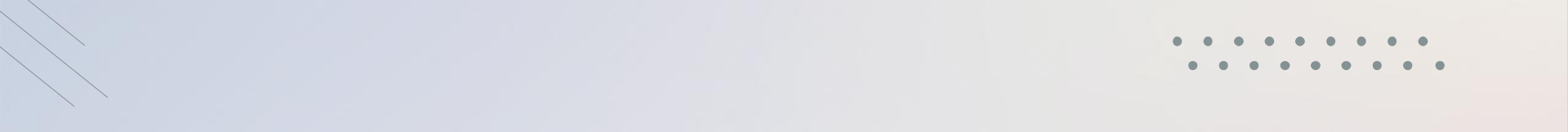
1

Software Qualities

Software Qualities

A quality attribute is a measurable or testable property of a system that is used to indicate how well the system satisfies the needs of its stakeholders beyond the basic function of the system.

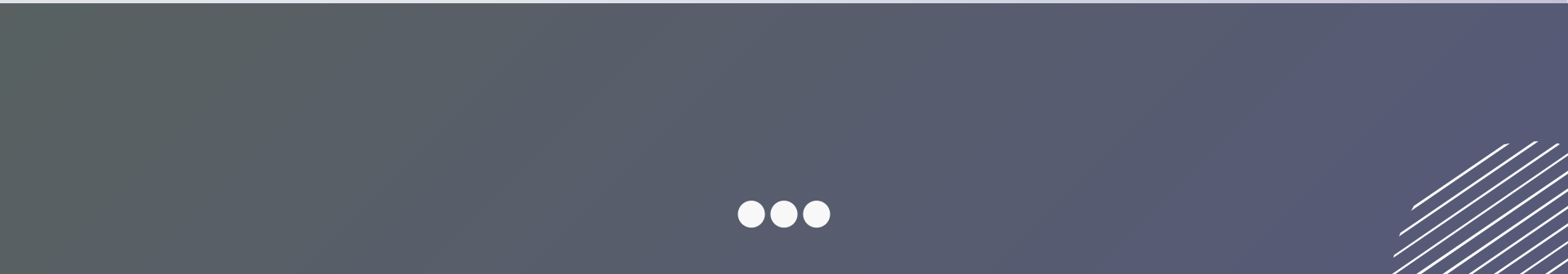




Software Qualities

vs.

*Non-functional
requirements*



Software Qualities

≈

*Non-functional
requirements*

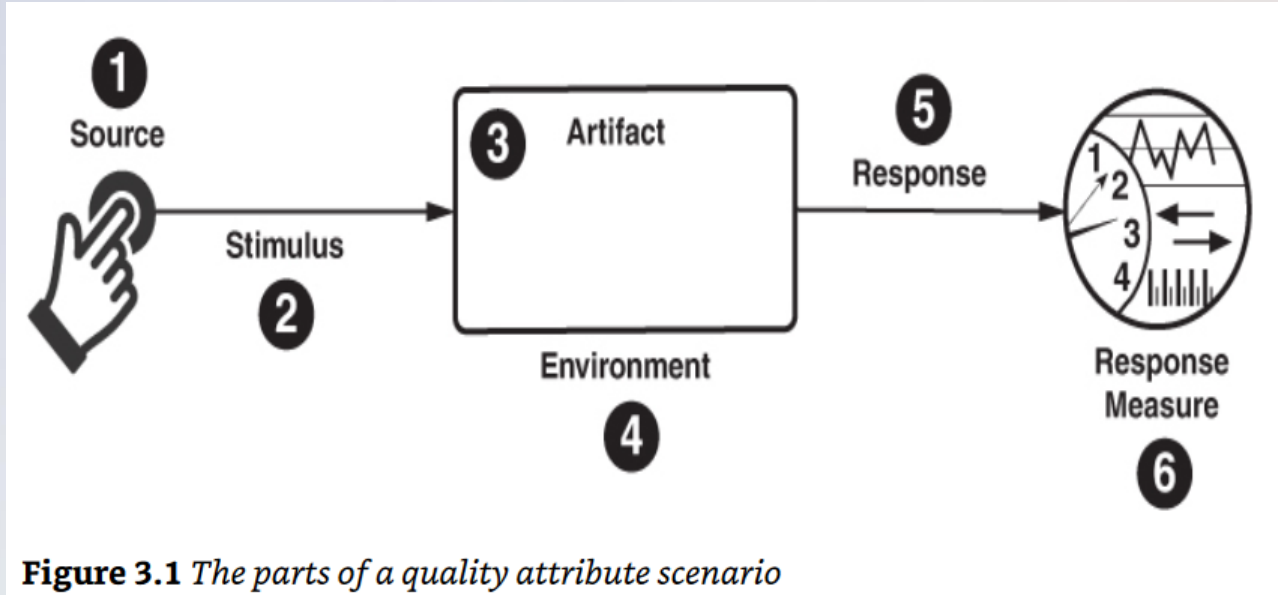


Functionality vs. Quality

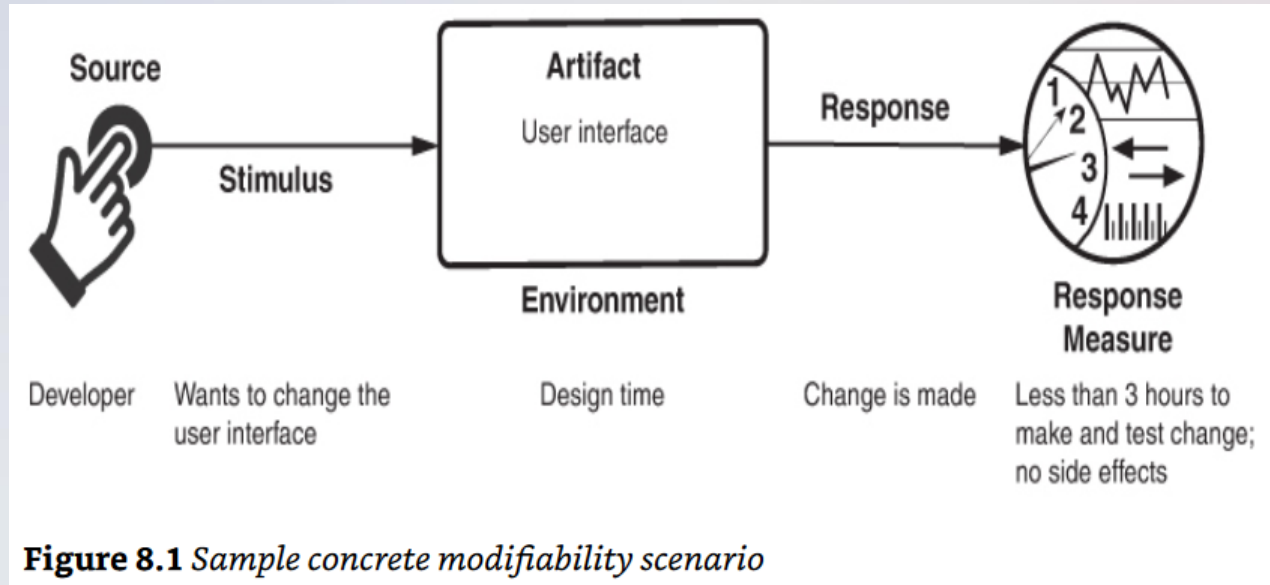
Functional Requirement	Quality Attribute
New accounts should have a user defined password.	Users should immediately change the system assigned password into a user defined one Security
Create a homepage where the user can see all their information	Homepage screen should load up in less than 0.2 seconds Performance
Create a form which would process and validate user's data	The form should be modular, as the required fields are highly dynamic Modifiability



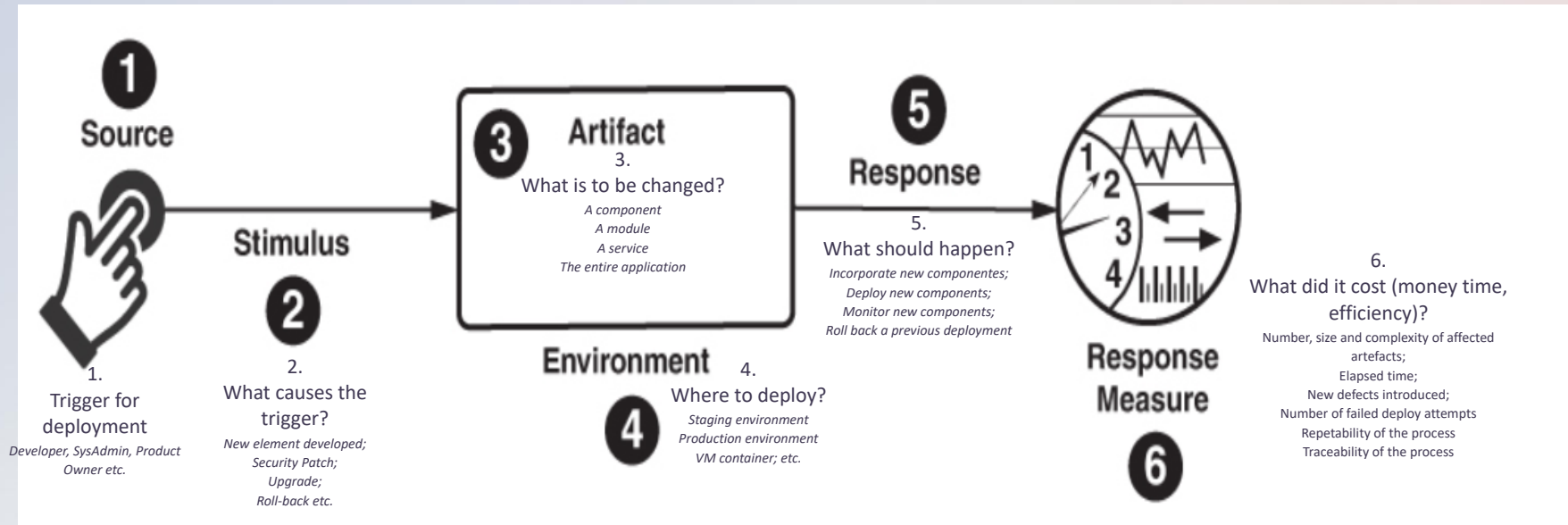
Quality Attribute Scenario



Example: Modifiability Quality Attribute Scenario

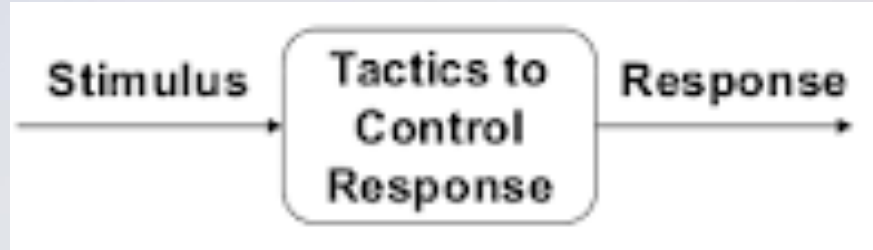


General Scenario



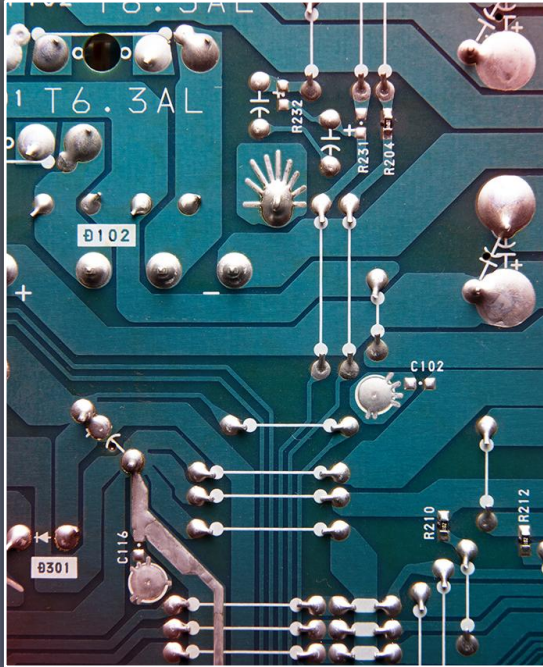
Tactics

Tactics are decisions implemented in order to achieve quality attribute scenarios



Deployability Tactics





2

Software Architecture

Defining Software Architecture



Geisel Library, University of California, San Diego¹



Bosco Verticale, Milan³



State Hermitage Museum, St. Petersburg²



Pagoda of Kaiyuan, Dingzhou⁴

¹<https://www.architecturaldigest.com/gallery/brutalist-buildings-on-college-campuses>

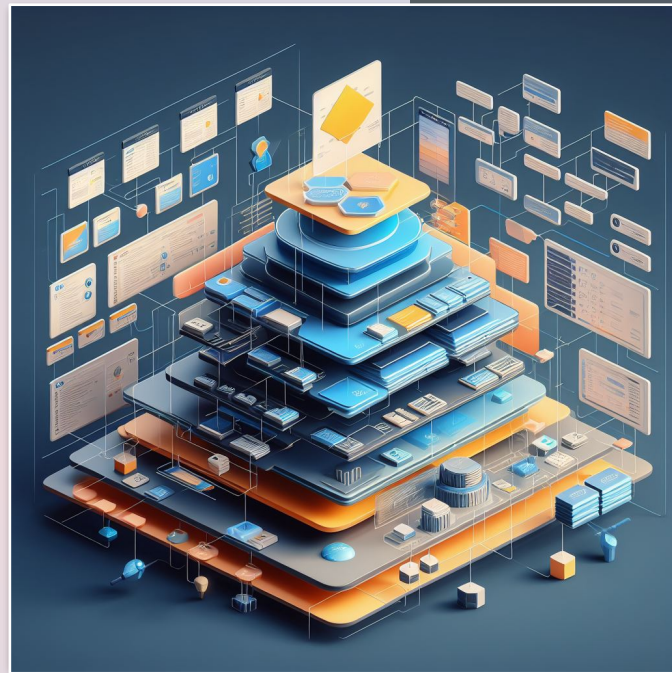
²<https://www.corinthia.com/st-petersburg/discover-st-petersburg/top-10-things-to-see-in-the-hermitage-collection/>

³<https://www.greenqueen.com.hk/urban-forest-of-the-future-how-to-turn-our-cities-into-treetopias/the-bosco-verticale-vertical-forest-in-milan-shutterstock/>

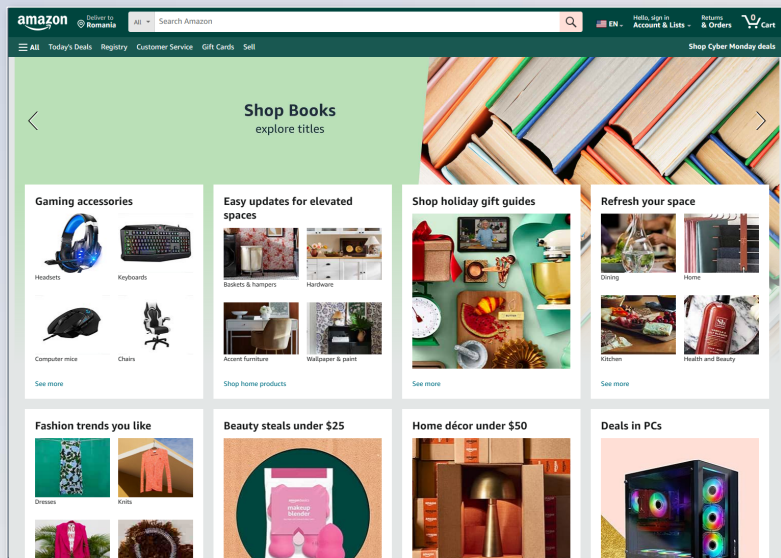
⁴https://www.123rf.com/photo_10521896_pagoda-of-kaiyuan.html

Defining Software Architecture

*“The set of structures needed to **reason** about the system, which comprise software **elements**, **relations** among them, and **properties** of both.”*



DALL-E 3 generated image of “Software Architecture”

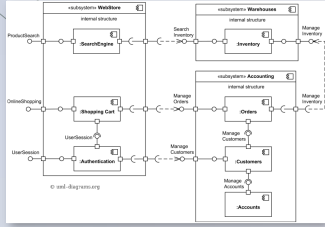


Amazon.com



Bosco Verticale, Milan

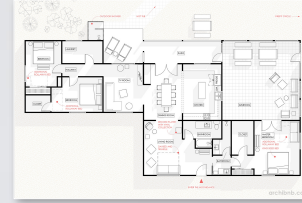




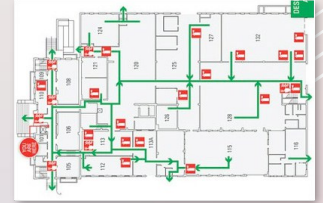
Component Diagram



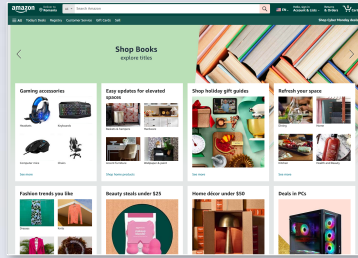
Application monitoring



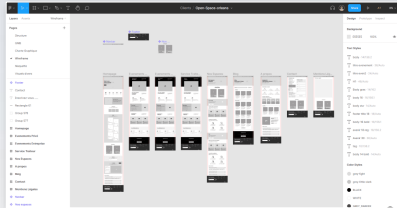
Floor layout plan



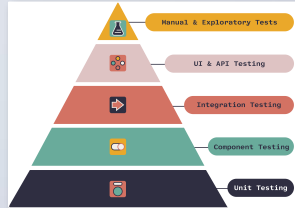
Safety Evacuation plan



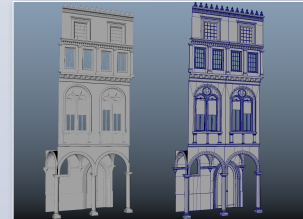
Amazon.com



Design & Wireframe



Software Testing



Building front



Earthquake testing

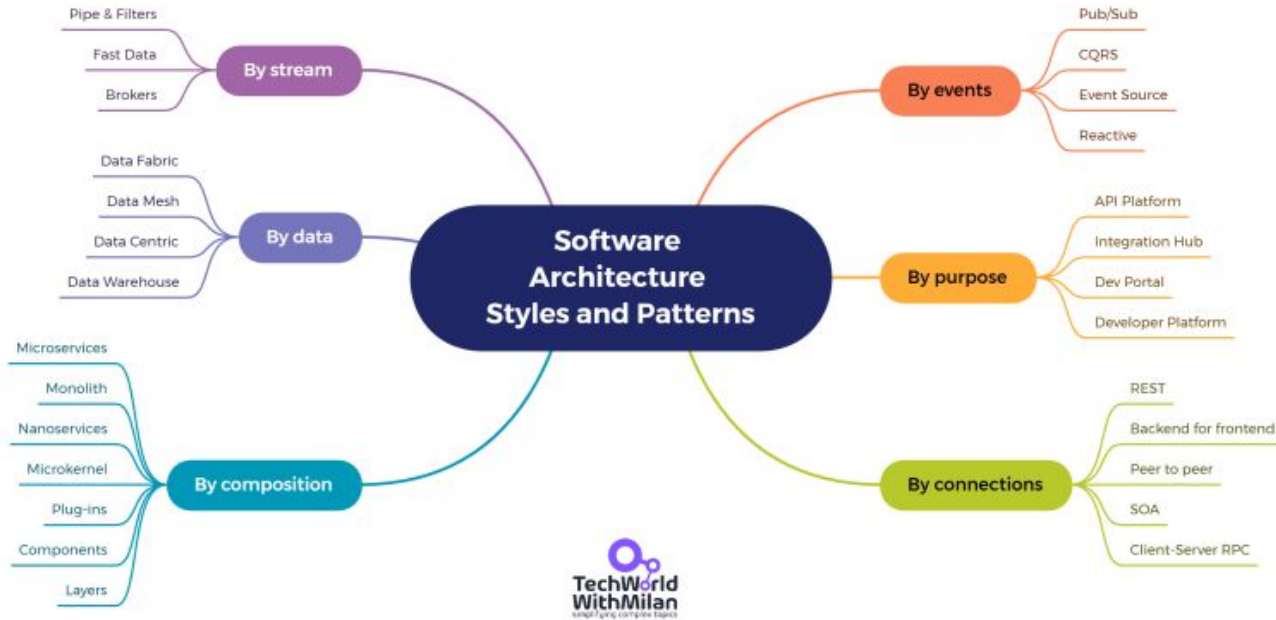
Architectural Patterns

Architectural Patterns are **reusable solutions** to common problems that provide a structural framework for software applications.

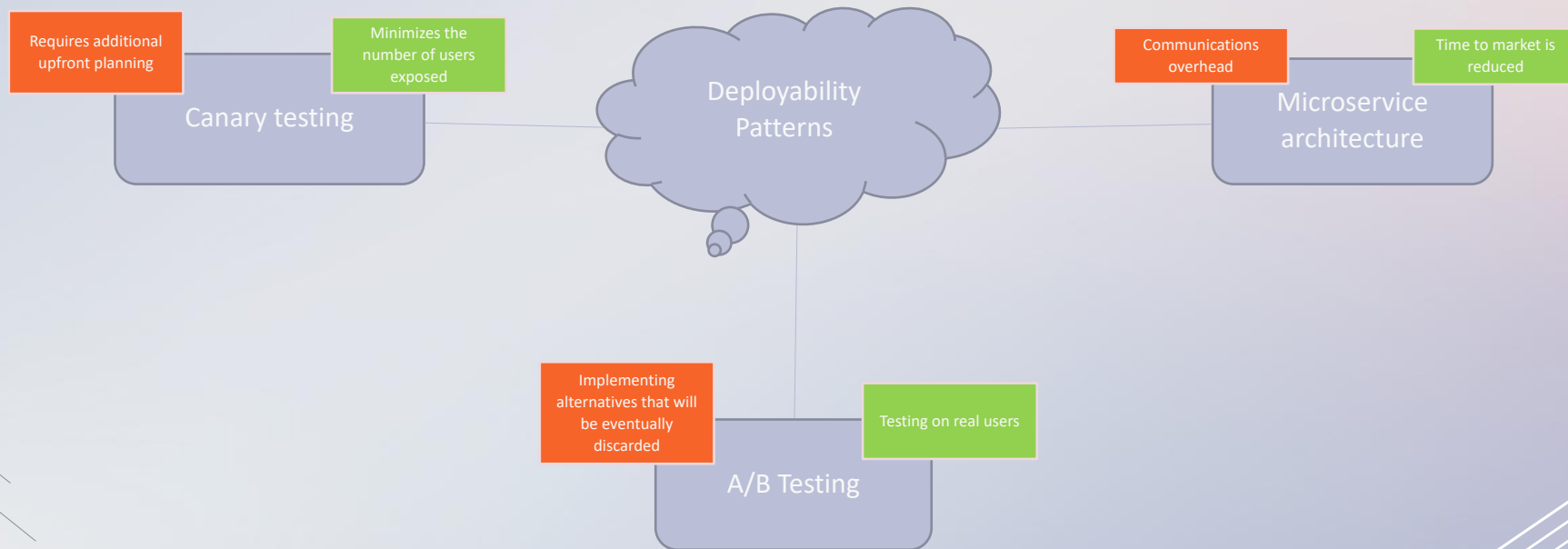
Pattern	Tactic
Solution that defines a fundamental architectural choice	Solution that solves a punctual requirement
Generally produces tradeoffs	A “best solution” can be identified



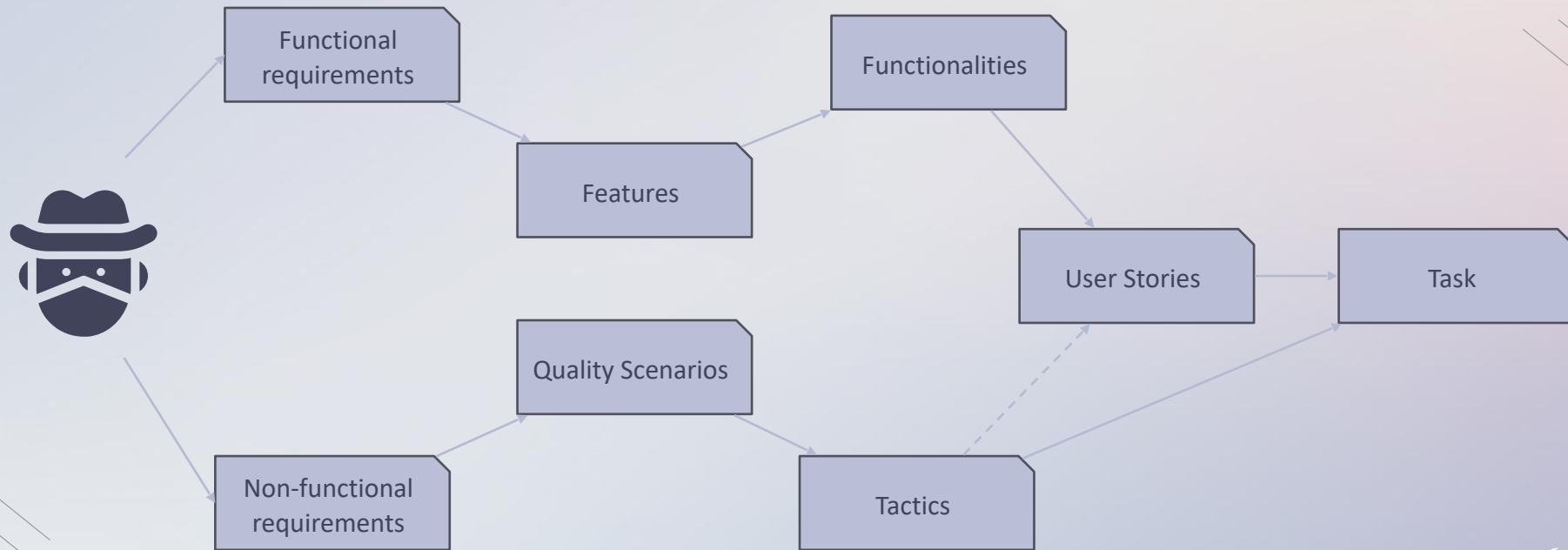
Software Architecture Categorization



Deployability Patterns

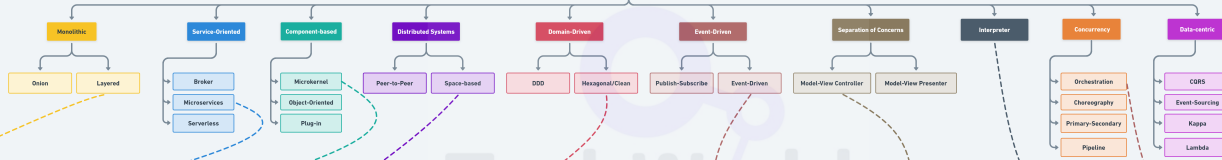


Putting it all together



Software Architecture Styles

Software Architecture Styles

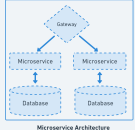


Separate software into logical layers



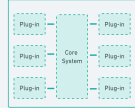
Layered Architecture

Divides an app into small, independent services that communicate through standard protocols



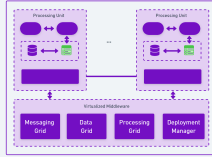
Microservice Architecture

Breaks the system into a minimal core and separate modules for extended functionalities



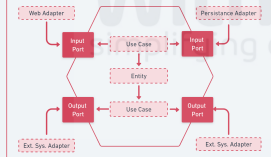
Microkernel Architecture

Split data and processes into distributed "spaces" to avoid data contention and achieve fault tolerance and scalability



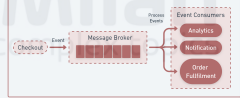
Space-based Architecture

Give separation of concerns by organizing application logic into a central core with interchangeable peripheral components



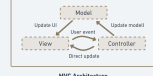
Hexagonal Architecture

Components perform actions in response to events or changes, rather than following a predetermined sequence



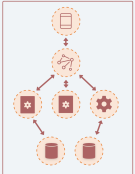
Event-Driven Architecture

Separate the concerns of data management, user interface and control flow



MVC Architecture

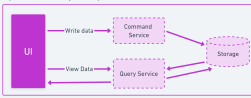
High-level language was used, which the interpreter then converted into executable code



Orchestration

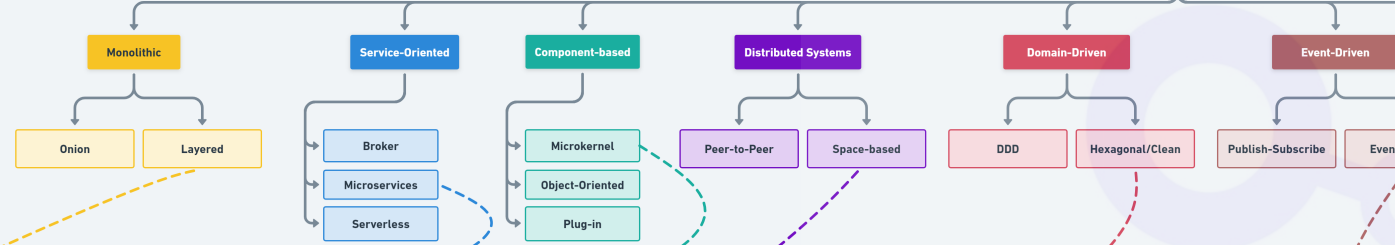
Orchestrator manages and coordinates tasks or workflows between multiple services or components, ensuring they work together seamlessly

Data modification and data retrieval are separated into distinct systems or processes

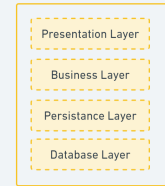


CQRS Architecture

Software Architecture Styles

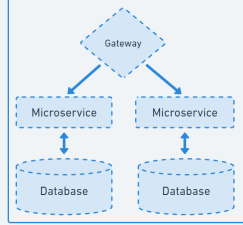


Separate software into logical layers



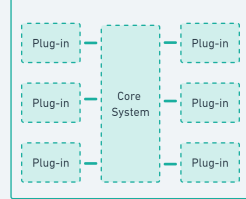
Layered Architecture

Divides an app into small, independent services that communicate through standard protocols



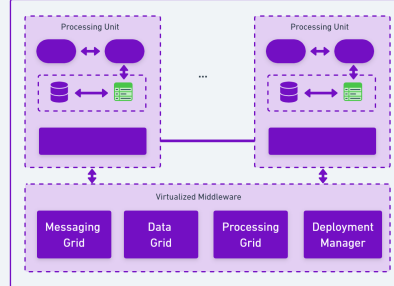
Microservice Architecture

Breaks the system into a minimal core and separate modules for extended functionalities.



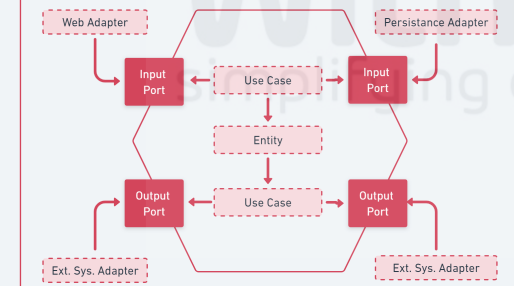
Microkernel Architecture

Split data and processes into distributed "spaces" to avoid data contention and achieve fault tolerance and scalability.



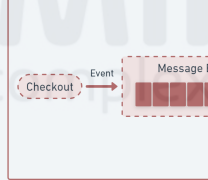
Space-based Architecture

Give separation of concerns by organizing application logic into a central core with interchangeable peripheral components



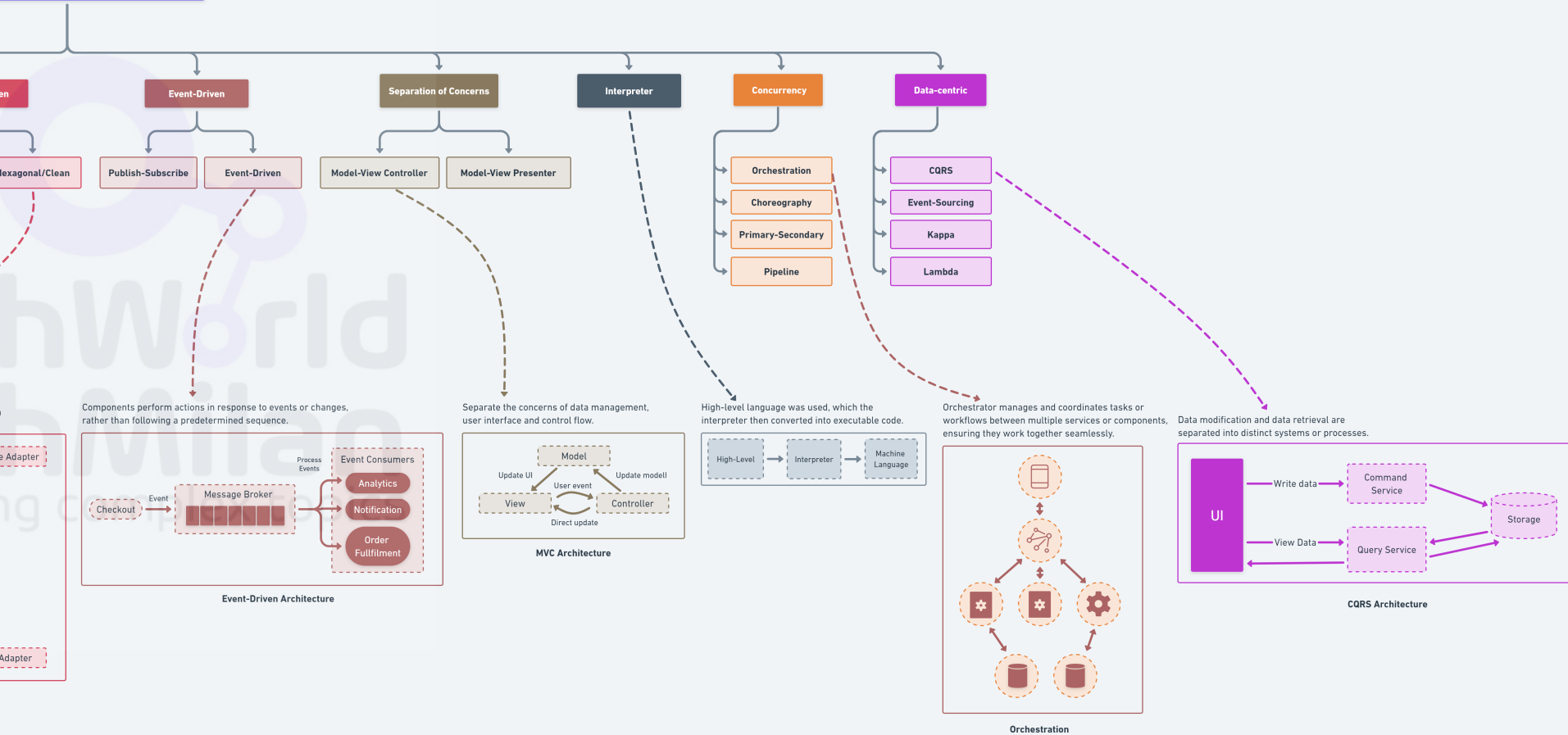
Hexagonal Architecture

Components perform actions in response to an event rather than following a predetermined flow



Event-driven Architecture

Architecture Styles



Non-functional system quality attributes

Table 4.2 Non-functional system quality attributes

Attribute	Key issue
Responsiveness	Does the system return results to users in a reasonable time?
Reliability	Do the system features behave as expected by both developers and users?
Availability	Can the system deliver its services when requested by users?
Security	Does the system protect itself and users' data from unauthorized attacks and intrusions?
Usability	Can system users access the features that they need and use them quickly and without errors?
Maintainability	Can the system be readily updated and new features added without undue costs?
Resilience	Can the system continue to deliver user services in the event of partial failure or external attack?

Let's think about this...



An airline wants to promote its fidelity programme by working together with a bank to provide branded credit cards with various benefits.

The bank will build a special internet banking app for these clients. What architectural challenges can you see for this app?

Choosing the architecture

1. Define functional and non-functional system requirements

2. Analyze architecture domains and viewpoints

3. Compare architectural styles

4. Chose an architecture and justify your choice

Kahoot!

Next time on ...

Course 5

Please read Chapter 3 from Brown – The C4 model for visualizing software architecture

And / or

Watch [Visualising software architecture with the C4 model - Simon Brown, Agile on the Beach 2019](#)